Transformer: 45 Interview Questions, Corrected Study Sheet

Source: Instagram carousel "Transformer 45 Most Important Interview Questions" by @techyy_bandaa (16 slides, posted September 19, 2026).

The deck does not actually deliver 45 distinct questions. It has 45 numbered slots (15 pages of 3), but question numbers 22-24 and 28-30 never appear on any slide, and numbers 25, 26 and 27 are each reused two or three times across pages 8-10 (25 and 26 repeated near-verbatim; the second use of "27" asks about the feed-forward network instead of padding, a different question under the same number). That leaves 39 distinct questions. Where a slide is a near-duplicate of an earlier one, this sheet gives the full answer once, under the first question number that carries it, and a short cross-referenced entry under the repeat's own number, listing only what is new or wrong on that particular slide, rather than repeating the same paragraphs and tables several times.

The cover slide's own architecture diagram is also wrong and worth flagging before anything else: it draws the encoder and decoder sharing one "self-attention -> multi-head attention -> feed forward" stack. In the real Transformer, each encoder layer has its own self-attention and feed-forward sublayers, and each decoder layer has masked (causal) self-attention, cross-attention into the encoder, and its own feed-forward sublayer (Vaswani et al. 2017, Figure 1 and section 3.2.3). Do not draw the cover version on a whiteboard.

Contents by theme

- **Attention mechanics:** Q1, Q6, Q8, Q9, Q20, Q33, Q34
- **Architecture families and components:** Q2, Q5, Q17, Q18, Q21, Q27 (FFN copy), Q32, Q43
- **Normalisation and residuals:** Q11, Q13, Q15, Q35
- **Masking:** Q12, Q26, Q27 (padding), Q31, Q39, Q42
- **Positional encoding:** Q3, Q4, Q7, Q10, Q16, Q19, Q25, Q36, Q38, Q44
- **Training and fine-tuning:** Q14, Q41
- **Inference and decoding:** Q45
- **Complexity and long context:** Q37
- **Limitations:** Q40
- **Not delivered by the deck:** Q22, Q23, Q24, Q28, Q29, Q30 (these numbers never appear on any slide)

Q1. What is the intuition behind the self-attention mechanism in Transformers? How does it help in capturing long-range dependencies?

Answer:

- Self-attention lets every token attend to every other token in the sequence and build a weighted representation from them.
- Each token is projected into a Query (Q), Key (K) and Value (V) vector. The attention score between two tokens comes from the similarity of their Q and K vectors; higher similarity gives more weight.

- It captures long-range dependencies because the path between any two tokens is one attention step (O(1)), not proportional to their distance, unlike an RNN which must carry information step by step.
- Self-attention is parallelizable across positions, which is why it trains faster than recurrent models on parallel hardware.

Diagram (self-attention, single head):

```
X (n x d_model) --Linear(W_Q)--> Q
X (n x d_model) --Linear(W_K)--> K
X (n x d_model) --Linear(W_V)--> V

Q, K, V --> Attention(QK^T / sqrt(d_k)) --> Softmax --> Output (n x d_model)
```

Formula: Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) V

Interview tip: be clear about the role of Q, K, V and why scaling is required.

Correction: the slide says scaling by sqrt(d_k) "prevents large dot products" and that self-attention is "more efficient for long sequences" than RNNs/CNNs. Both are wrong as stated.

- Scaling does not stop dot products from growing; it rescales them so their variance stays near 1 regardless of d_k. Without it, large-magnitude dot products push softmax into a region with near-zero gradients (Vaswani et al. 2017, section 3.2.1 footnote).
- Self-attention is not more efficient for long sequences: its per-layer cost is O(n^2 . d), against O(n . d^2) for a recurrent layer, so for long n it is more expensive per layer, not less. What it actually gains is a short (O(1)) path between any two positions and full parallelism across positions - that is why it trains faster in wall-clock time and learns long-range dependencies more easily (Vaswani et al. 2017, section 4 and Table 1).

Q2. Explain the difference between encoder-only, decoder-only and encoder-decoder Transformer architectures. Give one example of each.

Answer:

- Encoder-only: for understanding tasks (classification, sentence embeddings). Only an encoder stack.
- Decoder-only: for generation tasks (language modeling). Only a decoder stack with causal masking.
- Encoder-decoder: for sequence-to-sequence tasks (translation, summarization). Both stacks, with cross-attention.

Architecture	Used for	Example
Encoder-only	Understanding	BERT
Decoder-only	Generation	GPT
Encoder-Decoder	Seq2seq	T5, BART

Diagram:

```
Encoder-only (BERT)     Decoder-only (GPT)     Encoder-Decoder (T5)
Encoder Layer           Decoder Layer          Encoder Layer -> Decoder Layer
Encoder Layer           Decoder Layer          Encoder Layer -> Decoder Layer
   ...                     ...                    ...            ...
Encoder Layer           Decoder Layer          Encoder Layer -> Decoder Layer
Output                  Output                                  Output
```

Follow-up: Why does BERT use bidirectional attention while GPT uses causal masking?

This checks out. One addition for the follow-up: BERT was also trained with a second objective, Next Sentence Prediction, alongside masked language modelling (see Q43).

Q3. What is positional encoding in Transformers? Discuss different types of positional encoding and why it is needed.

Answer:

- Self-attention has no built-in notion of order (without positions it is permutation-equivariant), so positional encoding is added to give the model position information.
- Sinusoidal (original paper): sine and cosine functions at different frequencies.
- Learned: position embeddings trained as ordinary parameters.

Formula:

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

Type	Advantage (as printed)	Disadvantage
Sinusoidal	claimed to extrapolate to longer sequences	fixed pattern, less flexible
Learned	more expressive, learns patterns	no explicit position logic

Interview tip: be ready to explain why positional encoding is needed and how it works mathematically.

Correction: "allows extrapolation to longer sequences" (repeated on Q38 and Q44 too) overstates the paper's own claim, and later work contradicts it.

- The original paper only says the sinusoidal form "may allow the model to extrapolate to sequence lengths longer than the ones encountered during training" - a hope, not a demonstrated property.
- Measured directly, sinusoidal language models show "very weak extrapolation abilities" and cannot extrapolate more than a few dozen tokens past their training length. Purpose-built schemes such as ALiBi were created specifically to fix this (Press et al., "Train Short, Test Long").
- Say it as: "the paper hoped it would extrapolate; in practice it barely does, which is why ALiBi and rotary variants exist."

Also flagged here because it recurs later (Q4, Q16, Q36): "Learned: often better in practice" is not backed. The original paper found sinusoidal and learned encodings gave "nearly identical results" (Table 3, row E).

Q4. What is positional encoding in Transformers? Why is it needed and what are the different types of positional encoding?

Near-duplicate of Q3, same theory and formula. New on this slide: a comparison table.

Feature	Sinusoidal	Learned
Nature	fixed formula	trainable
Flexibility	less flexible	more flexible
Performance	good	"often better in practice"

Correction: see Q3 for the extrapolation point. "Learned ... often better in practice" is also unsupported; the paper found the two nearly identical.

Q5. Explain the difference between encoder-only, decoder-only and encoder-decoder Transformer architectures. Give one example of each.

Verbatim repeat of Q2 (same table, diagram and follow-up). No new content, no correction needed.

Q6. What are the common attention mechanisms used in Transformers? How do they differ and when are they used?

Answer:

- Self-attention: attends within the same sequence; used in the encoder, and in the decoder with causal masking.
- Cross-attention: decoder queries attend to encoder outputs; brings source information into the decoder.
- Multi-head attention: runs several attention heads in parallel to capture different relationship types at once.

Type	Query from	Key/Value from	Used in
Self-attention	same sequence	same sequence	encoder and decoder (masked)
Cross-attention	decoder	encoder	decoder
Multi-head	(multiple heads)	(multiple heads)	encoder and decoder

Diagram: Input -> [Head 1] [Head 2] ... [Head h] (parallel) -> Concatenate -> Linear -> Output

When to use: self-attention -> language modeling, classification; cross-attention -> seq2seq, translation; multi-head -> better representation learning.

Interview tip: be ready to explain the difference with a simple diagram.

Correction: the slide lists multi-head attention as a third "type" alongside self-attention and cross-attention, as if it were an alternative mechanism. It is not: multi-head is how self-attention or cross-attention is computed (as several smaller-dimensional copies run in parallel), not a separate kind of attention. The original Transformer uses multi-head attention in all three places it needs attention at all: encoder self-attention, decoder self-attention, and encoder-decoder (cross) attention (Vaswani et al. 2017, section 3.2.2).

Q7. What is the role of positional encoding in Transformers? Why is it necessary? What are the different types of positional encoding?

Near-duplicate of Q3/Q4. New here: a third type, "Relative - encodes relative distance between tokens", and a worked numeric example.

Position	Dim 0 (sin)	Dim 1 (cos)
0	0.0000	1.0000
1	0.8415	0.5403
2	0.9093	-0.4161
3	0.1411	-0.9900

This specific table is correct (verified by recomputation against the slide's own formula). See Q3 and Q44 for the errors elsewhere in the positional-encoding material.

Q8. Explain the self-attention mechanism in detail. What are the roles of Query, Key and Value vectors? How does scaled dot-product attention work?

Answer:

- Query: what a token is looking for. Key: what a token offers to be matched against. Value: the information actually aggregated.
- Steps: 1) compute Q.K^T; 2) scale by sqrt(d_k); 3) softmax to get attention weights; 4) multiply by V.

Formula: Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) V

Follow-up: difference between self-attention and cross-attention -> cross-attention takes K, V from the encoder and Q from the decoder.

Complexity (as printed): time O(n^2 . d), space O(n^2).

Slide code, C++ ("simplified"):

```
vector<vector<float>> scaledDotProduct
        (vector<vector<float>>& Q,
         vector<vector<float>>& K,
         vector<vector<float>>& V) {
    int n = Q.size();
    int d = Q[0].size();
    vector<vector<float>> scores(n, vector<float>(n));
    // Compute QK^T / sqrt(d)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            float dot = 0;
            for (int k = 0; k < d; k++) dot += Q[i][k]*K[j][k];
            scores[i][j] = dot / sqrt(d);
        }
    }
    // Apply softmax and multiply with V (omitted for brevity)
    return {};
}
```

Correction: this compiles but is useless as an answer to the question actually asked. It computes the scaled QK^T scores and then `return {};`, discarding the result. There is no softmax, V is never used, and no mask is applied. It also takes Q, K, V by non-const reference, so it cannot be called with temporaries. Also see Q1: scaling does not "prevent large dot products", it keeps the softmax input's variance stable.

Working version (Python):

```
import math, torch

def attention(q, k, v, mask=None):
    # q: (..., n, d_k)  k: (..., m, d_k)  v: (..., m, d_v)
    # mask: bool, broadcastable to (..., n, m); True = may attend
    scores = q @ k.transpose(-2, -1) / math.sqrt(q.size(-1))
    if mask is not None:
        scores = scores.masked_fill(~mask, float("-inf"))
    return torch.softmax(scores, dim=-1) @ v
```

Working version (C++), if the interview specifically wants C++:

```
#include <vector>
#include <cmath>
#include <algorithm>
using std::vector;

vector<vector<float>> scaledDotProduct(const vector<vector<float>>& Q,
                                       const vector<vector<float>>& K,
                                       const vector<vector<float>>& V) {
    size_t n = Q.size(), m = K.size(), d = Q[0].size(), dv = V[0].size();
    float scale = 1.0f / std::sqrt(static_cast<float>(d));
```

```
    vector<vector<float>> out(n, vector<float>(dv, 0.0f));
    vector<float> w(m);
    for (size_t i = 0; i < n; ++i) {
        float mx = -INFINITY;
        for (size_t j = 0; j < m; ++j) {
            float dot = 0.0f;
            for (size_t k = 0; k < d; ++k) dot += Q[i][k] * K[j][k];
            w[j] = dot * scale;
            mx = std::max(mx, w[j]);
        }
        float sum = 0.0f;
        for (size_t j = 0; j < m; ++j) { w[j] = std::exp(w[j] - mx); sum += w[j]; }
        for (size_t j = 0; j < m; ++j) {
            float a = w[j] / sum;
            for (size_t k = 0; k < dv; ++k) out[i][k] += a * V[j][k];
        }
    }
    return out;
}
```

This version never stores the full n x m score matrix as a separate un-normalized buffer for longer than one query row, and uses a numerically stable (max-subtracted) softmax.

Q9. What is multi-head attention? Why is it better than single-head attention? What happens if we use only one head?

Answer:

- Multiple heads let the model attend to information from different representation subspaces at different positions at the same time; different heads can specialize in different relationship types.
- Head outputs are concatenated and linearly projected back to d_model.

Aspect	Single-head	Multi-head
Heads	1	multiple
Representation	single subspace	multiple subspaces
Performance	limited	better

Follow-up: what happens with one head -> less expressive, may miss patterns.

Correction: the slide's table lists multi-head as "Higher" complexity than single-head, and an "edge case" claims multi-head "may overfit" on very small data. Neither holds up.

- With h heads each using d_k = d_model / h, total compute and parameter count stay about the same as one full-dimensional head: "the total computational cost is similar to that of single-head attention with full dimensionality" (Vaswani et al. 2017, section 3.2.2).

- There is no stated basis for the small-data-overfitting claim; parameter count does not grow with head count, so it is not the mechanism you would expect to cause overfitting.
- What actually degrades with fewer heads is quality, not cost: the paper's ablation (Table 3, row A) shows single-head attention scoring about 0.9 BLEU worse than 8 heads ("averaging inhibits this"), and quality also drops with too many heads.

Q10. What is the purpose of positional encoding in Transformers? What are the different types of positional encoding?

Near-duplicate of Q3/Q4/Q7. New table:

Type	How it works	Pros	Cons
Sinusoidal (fixed)	sine & cosine functions	generalizes, no extra params	may not be optimal for all tasks
Learned	learned embeddings	flexible	needs more data & parameters
Relative	encodes relative distance	good for long sequences	more complex to implement

IMPORTANT box: "BERT uses learned, T5 uses relative" - correct (BERT's position embeddings are learned, max length 512). See Q3 for the extrapolation correction, which this slide's table does not repeat directly but is part of the same recurring claim family.

Q11. What is the role of Layer Normalization in Transformers? How is it different from Batch Normalization?

(Title garbled on this slide by an apparent rendering glitch; wording taken from the clean repeat at Q13. Body content is legible.)

Answer:

- LayerNorm normalizes each token's feature vector independently (across the d_model dimension), not across the batch, so it does not depend on batch size.
- It stabilizes training and works with variable-length sequences and small batches, which is why it is preferred over BatchNorm in Transformers and RNNs.

Aspect	LayerNorm	BatchNorm
Normalizes across	features, per sample	batch, per feature
Works well with	variable batch sizes, sequences	large batches, CNNs
Used in	Transformers, RNNs	CNNs, vision models

Diagram: X -> Multi-Head Attention -> Add (+X) -> LayerNorm -> Feed Forward -> Add (+previous) -> LayerNorm -> Output

Follow-up: Why not use BatchNorm in Transformers? -> variable sequence lengths and small batch sizes make BatchNorm less effective.

Correction: "normalizes ... for each sample" is imprecise. In a Transformer, LayerNorm normalizes each token position's d_model-length vector separately - it is per token, not per whole sequence. The deck also never states the strongest practical reason to prefer LN: LayerNorm "uses statistics computed from input data in both training and evaluation modes," whereas BatchNorm relies on batch statistics during training and a running average at inference, so BatchNorm behaves differently at train vs test time in a way that interacts badly with variable-length, small-batch NLP data (PyTorch LayerNorm docs). Also worth knowing: LayerNorm learns a per-element scale and bias (gamma, beta); BatchNorm learns one scale/bias per channel.

Q12. How does attention masking work in Transformers? What are the different types of masks used?

(Title garbled on this slide; body content is fully legible and unambiguous.)

Answer:

- A mask blocks certain positions from being attended to (padding tokens, or future tokens for generation). The mask is added to the raw attention scores before softmax, so masked positions get near-zero probability.
- Padding mask: hides padding tokens in variable-length batches.
- Look-ahead (causal) mask: prevents a position from attending to future positions; used in decoders for generation.
- Combined mask: both together, common in seq2seq decoders.

How it works: 1) scores = QK^T / sqrt(d_k); 2) add mask (masked positions -> -infinity); 3) softmax; 4) weighted sum with V.

Look-ahead mask example (4 tokens, 0 = attend, -inf = masked):

	tok1	**tok2**	**tok3**	**tok4**
tok1	0	-inf	-inf	-inf
tok2	0	0	-inf	-inf
tok3	0	0	0	-inf
tok4	0	0	0	0

This checks out; no correction needed. Practical note worth keeping: the standard fill value is -inf (PyTorch's reference implementation uses this). A slide elsewhere in the deck (Q39) uses -1e9 as the fill value - that is fine in float32 but overflows float16, whose maximum representable value is about 65504, so match the fill value to the tensor's dtype.

Q13. What is the role of Layer Normalization in Transformers? How is it different from Batch Normalization?

Clean repeat of Q11 - same content, same table. No new correction beyond Q11's.

Q14. What is the purpose of Dropout in Transformers? How does it work and what are its benefits?

Answer:

- Dropout randomly zeroes a fraction p of activations during training, to prevent co-adaptation and overfitting.
- Applied after the attention output, after the feed-forward layer, and to the embeddings plus positional encodings.

How it works: 1) pick a rate p (e.g. 0.1); 2) sample a Bernoulli mask; 3) zero activations where mask = 0; 4) scale the kept activations by 1/(1-p) - this scaling happens during training (inverted dropout).

Worked example (p = 0.1): input [1.0, 2.0, 3.0, 4.0], mask [1, 0, 1, 1] -> [1.11, 0, 3.33, 4.44].

Typical rates: attention, FFN, embedding and residual dropout all around 0.1.

Follow-up: what happens with too high/low a dropout rate; why dropout can beat L2 regularization in some cases.

Correction: the slide's interview tip says "during inference, dropout is turned off and activations are scaled appropriately" - the scaling does not happen at inference. With inverted dropout (what the slide's own step 4 describes, and what PyTorch implements), the 1/(1-p) scaling is applied during training; at evaluation time dropout is simply the identity function and nothing is rescaled. The tip contradicts the slide's own worked steps.

Q15. Explain the concept of Residual Connections in Transformers. Why are they important and how do they help in training deep models?

Answer:

- A residual (skip) connection adds a layer's input to its output: y = F(x) + x.
- Used around both the multi-head attention sublayer and the feed-forward sublayer.
- Helps gradients flow through many stacked layers and prevents them from vanishing, allowing much deeper models to train.

Diagram: X -> [Multi-Head Attention] -> Add(+X) -> LayerNorm -> [Feed Forward] -> Add(+previous) -> LayerNorm -> Output

Aspect	With residual	Without
Gradient flow	better	poor
Training deep nets	easier	difficult
Convergence	faster	slower

Follow-up: what happens if we remove residual connections; why add LayerNorm after the residual.

Correction: the slide claims residual connections are "usually followed by Layer Normalization (in Pre-LN or Post-LN variants)" - that only describes Post-LN. In the original Transformer (Post-LN), the block is LayerNorm(x + Sublayer(x)) - norm after the residual add, which matches the slide. In Pre-LN, the norm moves inside the residual branch, before the sublayer: x + Sublayer(LayerNorm(x)) - here the residual is not followed by a norm. This matters in practice: Post-LN needs a learning-rate warm-up to avoid large gradients near the output at initialization, while Pre-LN trains stably without it ("On Layer Normalization in the Transformer Architecture", Xiong et al.).

Q16. What is the purpose of positional encoding in Transformers? What are the different types of positional encodings?

Near-duplicate of Q3/Q4/Q7/Q10. Adds a three-icon diagram (sinusoidal wave, learned grid, relative distance arrow) and a three-column feature comparison table (Sinusoidal / Learned / Relative). No new claims beyond what is already corrected under Q3.

Q17. Explain the difference between encoder-only, decoder-only and encoder-decoder Transformer architectures. Give one example of each.

(Title garbled on the slide, e.g. "decoder-only" rendered as "aecoder-bnly"; body content and table are unambiguous and identical to Q2.) Adds one new box: "Encoder-only -> no generation ability. Decoder-only -> needs causal mask. Encoder-decoder -> needs cross-attention." Accurate, no correction needed.

Q18. What is the role of the feed-forward network (FFN) in Transformers? How does it work and why is it used?

Answer:

- The FFN is applied to each token position independently and identically (position-wise): two linear layers with a non-linearity between them.
- Adds non-linearity and capacity after the attention sublayer; applied with its own residual connection and LayerNorm.

Formula: FFN(x) = W2 . sigma(W1 x + b1) + b2, usually d_model -> d_ff -> d_model (512 -> 2048 -> 512 in the base model).

Diagram: Input (seq_len, d_model) -> Linear(d_model -> d_ff) -> activation -> Linear(d_ff -> d_model) -> Output (seq_len, d_model)

Complexity (as printed): time O(N . d_model . d_ff).

Slide code, C++ ("simplified", written against an untyped Linear/Tensor-style API):

```
struct FFN {
    Linear fc1, fc2;
    FFN(int d_model, int d_ff) {
        fc1 = Linear(d_model, d_ff);
        fc2 = Linear(d_ff, d_model);
    }
    Tensor forward(const Tensor& x) {
        x = fc1(x);
        x = gelu(x);   // or relu(x)
        x = fc2(x);
        return x;
    }
};
```

Correction:

- Does not compile: `forward` takes `x` by `const Tensor&` and then assigns to it (`x = fc1(x);`), which is illegal for a const reference. `Linear`, `Tensor` and `gelu` are also undeclared - this is pseudocode shaped like libtorch, not compilable C++. The Linear -> activation -> Linear logic itself is correct.
- The slide's "usually ReLU or GELU" is not what the original paper used: Vaswani et al. specify a ReLU between the two linear layers. GELU became common in later models (BERT, GPT-2), but presenting it as the original default is inaccurate.
- The stated space complexity "O(N . d_model + d_ff)" ignores the actual intermediate activation (N x d_ff) and the roughly 2 x d_model x d_ff weights. The time complexity O(N . d_model . d_ff) is correct.

Working version (Python):

```
import torch

class FFN(torch.nn.Module):
    def __init__(self, d_model=512, d_ff=2048):
        super().__init__()
        self.fc1 = torch.nn.Linear(d_model, d_ff)
        self.fc2 = torch.nn.Linear(d_ff, d_model)
    def forward(self, x):  # x: (batch, seq, d_model)
        return self.fc2(torch.nn.functional.gelu(self.fc1(x)))
```

Q19. What is the purpose of positional encoding in Transformers? How does it work and what are the different types?

Near-duplicate, content identical in substance to Q7 (same formula, same tables). No new content or correction beyond what is already noted.

Q20. What is the role of self-attention in Transformers? Why is it called "self-attention" and how does it help the model?

Answer:

- Called "self"-attention because Q, K and V all come from the same sequence, with no external memory or second sequence involved - contrast with cross-attention (Q34).
- Helps the model capture long-range dependencies, handle variable-length input, and build contextual token representations.

Follow-up: how is it different from RNNs/CNNs -> RNNs are sequential, CNNs use local windows; self-attention connects any two positions directly.

Formula: Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) V

Advantages listed: captures long-range dependencies, more parallelizable than RNNs, handles variable-length input well. This entry does not repeat the "more efficient for long sequences" overreach from Q1, and checks out as printed.

Q21. What is the difference between encoder-only, decoder-only and encoder-decoder Transformer architectures? Give one example of each.

(Title heavily garbled/scrambled on this slide; content is the same family as Q2/Q5/Q17.) Adds a "Key Differences" table:

Aspect	Encoder-only	Decoder-only	Encoder-Decoder
Attention	bidirectional	causal	encoder bidirectional, decoder causal
Objective	understanding	generation	seq2seq
Example	BERT	GPT	T5, BART

IMPORTANT: encoder-decoder models are more complex but better for translation; encoder-only models are more efficient for classification/embedding. Accurate, no correction needed.

(Question numbers 22, 23 and 24 do not appear on any slide in the deck.)

Q25. What is the role of positional encoding in Transformers? Why can't we just use the model without positional encoding?

Answer:

- Self-attention by itself has no built-in notion of order.
- Positional encoding is added to the token embeddings (fixed or learned) before the first layer, so the model can use sequence order.
- Without it, permuting the input tokens permutes the model's per-token outputs the same way, but does not otherwise change them - so the model cannot tell "dog bites man" from "man bites dog" from position alone.

Type	How it works
Sinusoidal (fixed)	sine & cosine functions of different frequencies
Learned	position embeddings learned during training
Relative	encodes relative distance between tokens (e.g. T5, DeBERTa)

Diagram: Input tokens -> Token embeddings -> (+ Positional encoding) -> input to Transformer

(This slide is repeated near-verbatim two more times on the following pages, with minor wording and table differences but the same substance and the same errors.)

Correction: two claims here do not hold up.

- "Transformers ... are permutation invariant" is imprecise. Without positional information, self-attention is permutation-**equivariant**, not invariant: permuting the input tokens permutes the per-token outputs in the same way, rather than leaving the output unchanged. Only a pooled/aggregated output would stay identical.
- The interview tip on this slide says positional encoding "is not about giving the model absolute position" - this contradicts the deck's own answer here and elsewhere: sinusoidal and learned encodings are both absolute-position schemes; only the "Relative" type encodes distance between tokens rather than absolute position.
- Worth knowing for the interview: a causal (decoder-only) language model can learn a fair amount of positional information even with no positional encoding at all, because the causal mask itself lets it count preceding tokens (Haviv et al., "Transformer Language Models without Positional Encodings Still Learn Positional Information"). So "cannot know order without positional encoding" really applies to bidirectional encoders, not causal decoders.

Q26. What is the difference between masked self-attention and causal self-attention? When is each used?

Slide's answer, as printed:

- "Masked self-attention allows each token to attend to all tokens in the input sequence (including future tokens) ... used in tasks like BERT."
- "Causal self-attention (look-ahead masking) restricts each token to attend only to previous tokens ... used in autoregressive models like GPT."

Feature	"Masked" (per slide)	Causal
Visible tokens	all tokens (bidirectional)	past + current
Mask type	none (or padding mask)	look-ahead mask
Used in	BERT, RoBERTa	GPT, LLaMA

Diagram as printed: two 4x4 grids - "Masked Self-Attention" shown as all 1s (fully dense); "Causal Self-Attention" shown as the standard lower-triangular pattern.

(This same question, table, and mostly this diagram repeat verbatim two more times later in the deck; a related but differently-framed version appears again as Q31.)

Correction: this is the single biggest error in the deck, and it appears on four separate slides (here, twice more as repeats, and again as Q31). "Masked self-attention" and "causal self-attention" are standard synonyms for the same mechanism, not two different ones.

- The original Transformer paper names the decoder's first sublayer "Masked Multi-Head Attention", masking it "to prevent positions from attending to subsequent positions" by "masking out (setting to -inf) all values in the input of the softmax which correspond to illegal connections" (Vaswani et al. 2017, section 3.2.3). That is exactly the causal/look-ahead mask - in the paper, "masked attention" means causal attention.
- BERT's self-attention is not masked at all (aside from ordinary padding masking) - it is full bidirectional attention, "jointly conditioning on both left and right context in all layers" (Devlin et al. 2018, section 3.1). The word "masked" in BERT refers to the Masked Language Model training objective, which masks 15% of input tokens (80% -> [MASK], 10% -> random token, 10% unchanged) - it masks tokens in the input, not entries of the attention matrix.
- Correct framing: masked self-attention = causal self-attention = the decoder's look-ahead mask. BERT uses unmasked, bidirectional self-attention; its "masking" is about the MLM training objective, not the attention mechanism. Given that, the "all 1s" grid in the diagram describes ordinary bidirectional attention (as in BERT) and should not be labeled "masked".

Follow-up: can we use causal attention in BERT? What other mask types exist? How does masking affect training vs inference?

Q27 (as first asked, pages 8-9 - variable-length sequences and padding). How does the Transformer handle variable-length sequences? What are the common approaches?

(Title garbled on its first appearance; wording taken from the clearer of its two near-identical copies.)

Answer:

- Real batches contain sequences of different lengths, so shorter sequences are padded to the batch's max length.
- A padding mask (1 = real token, 0 = pad) is created so the model never attends to padding.
- Positional encoding and any causal mask are applied alongside the padding mask.

Example: seq1 [10,25,66] padded to [10,25,66,0], mask [1,1,1,0]; seq2 [8,14,7,6], mask [1,1,1,1]; seq3 [5,9] padded [5,9,0,0], mask [1,1,0,0].

Slide code, C++:

```
vector<vector<int>> createMask(const vector<int>& len, int maxLen) {
    vector<vector<int>> mask(len.size(), vector<int>(maxLen, 0));
    for (int i = 0; i < len.size(); i++) {
        for (int j = 0; j < len[i]; j++) {
            mask[i][j] = 1;
        }
    }
    return mask;
}
```

This one is largely usable as written: it compiles (aside from a signed/unsigned comparison warning) and produces the correct 1/0 mask, O(n*m) time and space as stated. Its one real bug: if any `len[i] > maxLen`, the inner loop writes past the end of that row (undefined behaviour). Fix by bounding the loop: `for (int j = 0; j < min(len[i], maxLen); j++)`.

Correction: the framing that "Transformers require fixed-length/padded sequences" is backwards. Self-attention works for any sequence length up to the positional limit; padding exists only so a batch of different-length sequences can be stacked into one rectangular tensor for efficient GPU computation, not because the architecture itself needs fixed length.

Q27 (as re-asked on page 10 - a different question reusing the same number - the feed-forward network). How does the Transformer's feed-forward network (FFN) work?

(The rest of this slide's title is corrupted on the image and not reconstructible.) Content matches Q18 almost exactly: same FFN structure, formula and diagram, and the same C++ "simplified" pseudocode as Q18's - see Q18 for the full correction (does not compile; ReLU vs GELU; understated space complexity). One extra issue specific to this slide: its interview tip is "Remember to handle padding and masking together for correct attention in both training and inference" - copied verbatim from the padding-mask question a few slides earlier and unrelated to the FFN. Treat it as a copy-paste error, not FFN advice.

(Question numbers 28, 29 and 30 do not appear on any slide in the deck.)

Q31. What is the difference between causal self-attention and masked self-attention? When are they used?

This slide frames it differently from Q26: "masked self-attention" is described as a general term for any attention mask, with causal attention as "a specific type of masked self-attention where the mask is structured to block future positions." Its diagram draws two 5x5 grids: "Masked Self-Attention (BERT)" as a scattered mix of 1s and 0s, and "Causal Self-Attention (GPT)" as the standard lower-triangular pattern.

Correction: framing causal attention as a special case of a general mask is a defensible way to talk about masking mechanics, but it is inconsistent with Q26's framing a few pages earlier (there, "masked" = bidirectional/unmasked) - the deck contradicts itself across its own repeats. This slide's diagram also invents a scattered-zero attention matrix for BERT; BERT's attention is fully dense (bidirectional) apart from ordinary padding, it does not zero out arbitrary entries. See the full correction under Q26.

Q32. How does the Position-wise feed-forward network (FFN) work? Why is it used in Transformers?

Third appearance of the FFN question (see Q18, and the page-10 Q27). Adds a "Typical Dimensions" box: d_model = 512, d_ff = 2048, d_ff = 4 x d_model - correct, matches the base Transformer configuration.

Slide code, C++:

```
vector<vector<float>> ff(const vector<vector<float>>& x) {
    int seq_len = x.size();
    int d_model = x[0].size();
    int d_ff = 4 * d_model;
    vector<vector<float>> out(seq_len, vector<float>(d_model));
    for (int i = 0; i < seq_len; i++) {
        for (int j = 0; j < d_model; j++) {
            float h1 = x[i][j] * W1[j];
            float h2 = max(0.0f, h1); // ReLU
            out[i][j] = h2 * W2[j];
        }
    }
    return out;
}
```

Correction: does not compile (`W1`, `W2` are undeclared), and the logic is wrong even if fixed up: it treats W1/W2 as per-feature scalars and never actually projects into the d_ff-wide space it computes (`d_ff`) and then ignores. A real FFN mixes across all d_model features to produce each of the d_ff intermediate values (a matrix multiply, not an elementwise scale), then mixes back the same way on the second layer. See the working Python version under Q18.

Q33. What are the advantages and disadvantages of multi-head attention? Why is it better than single-head attention?

(Title garbled on the slide as "posHead attention"; the body text says "multi-head" throughout, so that is the intended topic.)

Advantages: captures multiple relationship types in parallel, improves stability, better representation learning.

Disadvantages (as printed): "higher computational cost", "more memory usage", "slightly more complex implementation".

Edge case (as printed): "if the number of heads does not divide d_model, we use padding or adjust head size."

Correction: as with Q9, "higher computational cost" is not accurate - total compute and parameter count are similar to single-head attention at full dimensionality, since each of the h heads works in a proportionally smaller d_k = d_model / h (Vaswani et al. 2017, section 3.2.2). The real-world overhead multi-head attention adds is implementation overhead (splitting, concatenating), not fundamentally higher FLOPs or parameters. The "padding or adjust head size" edge case is not a standard technique either - in practice you simply choose a head count that divides d_model, which frameworks require: PyTorch's MultiheadAttention docs state embed_dim "will be split across num_heads (i.e. each head will have dimension embed_dim // num_heads)".

Q34. What is the difference between self-attention and cross-attention? When is cross-attention used?

Answer:

- Self-attention: Q, K, V all come from the same sequence.
- Cross-attention: Q comes from the decoder (target), K and V come from the encoder (source).

	Self-attention	**Cross-attention**
Source of Q, K, V	same sequence	different sequences
Used in	encoder/decoder self-attention	decoder (encoder-decoder models)
Purpose	intra-sequence context	attend to external context

When used: translation, summarization, image captioning, question answering.

Interview tip: cross-attention is essential in sequence-to-sequence tasks. Accurate as printed; no correction needed.

Q35. What is the difference between Layer Normalization and Batch Normalization in Transformers? Why is it used instead of batch

normalization?

(Title heavily garbled on the slide, literally rendering as "What is the diffferemnen inTransformers? Whl it used instead of batch normalization?"; the body content is legible and matches Q11/Q13's topic, and reconstructed wording above reflects that body content, not a guess at the garbled text itself.)

Adds the explicit LayerNorm formula, computed per token:

```
x_hat = (x - mu) / sqrt(sigma^2 + eps)
y = gamma * x_hat + beta
```

where mu, sigma^2 are the mean and variance across features for that token, and gamma, beta are learnable.

Correction: the formula and "for each token" framing here are correct, and clearer than Q11/Q13's "for each sample" wording. See Q11 for the fuller correction (train/eval statistics, per-element affine parameters) that this slide does not mention either.

Q36. How does sinusoidal positional encoding work in Transformers? What are the different types of positional encoding?

(Title garbled and not reconstructible beyond its evident topic, which the legible body content makes clear.) Adds a line-chart diagram of sine/cosine waves at decreasing frequency across positions 0-80, and repeats the same three-type comparison table as Q10/Q38. No new claims beyond what is already corrected under Q3 (extrapolation) and Q4 (learned-is-often-better).

Q37. How does the Transformer handle long sequences? What are the techniques used to improve its efficiency?

Answer:

- Standard self-attention is O(n^2) in sequence length, which gets expensive for long sequences.
- Sparse attention and local (windowed) attention reduce this to about O(nk) for a fixed window/pattern size k.
- Longformer and BigBird are named examples of structured sparse-attention models.
- FlashAttention and gradient checkpointing are named as further techniques.

Diagram: Long Sequence -> [Efficient Attention: Sparse, Local, Longformer, BigBird] and [Optimization: FlashAttention, Gradient Checkpointing, Mixed Precision]

Follow-up: max context length of GPT-3/4; how RoPE helps with long sequences; why linear attention is less commonly used.

Correction: the description of FlashAttention, "optimizes memory and computation on GPU" / "improves GPU memory bandwidth", is imprecise. FlashAttention is an exact (not approximate) attention algorithm that

is IO-aware: it tiles the computation "to reduce the number of memory reads/writes between GPU high bandwidth memory (HBM) and GPU on-chip SRAM" (Dao et al., FlashAttention). It changes how much data moves between memory levels and reduces wall-clock time and memory use; it does not reduce the underlying O(n^2) FLOP count, and it does not change the hardware's bandwidth. Gradient checkpointing and mixed precision, also listed here, are general memory-saving training techniques, not specifically long-context methods.

Also worth adding for the interview, since this answer omits it entirely: RoPE-based position scaling and ALiBi (named only in this slide's own follow-up), and the KV-cache, which is usually the actual memory bottleneck for long-context inference. See "What this deck does not cover" below.

Q38. What is the Role of Positional encoding in Transformers? How do different types compare?

Near-duplicate of Q3/Q10/Q36. Adds a "dog bites man" vs "man bites dog" motivating example and repeats the three-type comparison table.

Correction: see Q3 for the extrapolation correction, which this slide repeats ("generalizes well, works for longer sequences"). This slide adds a new, unsupported claim: "Learned embeddings are better for small datasets" - which directly contradicts the deck's own Q10 table, listing "needs more data & parameters" as learned encoding's con. Neither claim is sourced; treat both as unverified folklore rather than settled fact.

Q39. How does the Transformer handle variable-length sequences? What techniques are used for correct batching and masking?

Second appearance of the variable-length/masking question (see Q27). Adds a worked table: tokens "I, love, NLP, <pad>, <pad>", token IDs [101, 202, 303, 0, 0], padding mask [1, 1, 1, 0, 0], and a "causal mask" shown as one flat row of five 1s.

Correction:

- The causal mask here is drawn as a single row of all 1s. A causal mask is an n x n lower-triangular matrix (as the deck's own Q12 correctly shows), not a single row - a single row cannot express "position i cannot see position i+1". This table also does not show the causal mask interacting with the padding positions.
- The slide says "In PyTorch, use attention_mask and key_padding_mask." The actual PyTorch argument name is `attn_mask`, not `attention_mask` (`attention_mask` is Hugging Face's naming convention, not PyTorch's). The boolean polarity also differs by API, which is a real interview trap: in `torch.nn.MultiheadAttention`, `True` in `attn_mask` or `key_padding_mask` means "do not allow attention / ignore this key"; in `torch.nn.functional.scaled_dot_product_attention`, `True` means the element "should take part in attention" - the opposite polarity; Hugging Face's `attention_mask` uses `1` for a real, attendable token. Check the polarity for whichever API is actually being used rather than assuming.

Q40. What are the advantages and disadvantages of using Transformers in sequence modeling? When might other models be better?

Advantages: long-range dependencies via self-attention, parallelizable training, strong performance at scale, flexible across modalities.

Disadvantages: O(n^2) attention cost, needs large amounts of data, needs positional encoding (unlike RNNs), can be overkill for short sequences.

Aspect	Transformer	RNN/LSTM	CNN
Long-range deps	excellent	hard to learn	limited
Parallelization	yes	no	yes
Training speed	fast	slow	fast
Memory usage	high (O(n^2))	low	moderate
Best use case	large data, long sequences	short sequences, small data	local patterns (e.g. vision)

When others are better: RNN/LSTM for very short sequences, streaming data, low compute; CNN when local features matter more (images, speech) or training data is limited.

This is a reasonable, defensible summary and needs no correction, though "RNN better for small data" is a common heuristic rather than a specifically cited result.

Q41. How does prefix tuning / prompt tuning work in Transformers? What are they used for?

(Title garbled on the slide; reconstructed from the clearly legible body content, which is unambiguously about these two fine-tuning methods.)

Slide's answer, as printed: "Prefix tuning adds trainable 'virtual tokens' (prefix) to the input sequence. Only the prefix parameters are trained, while the original model weights are kept frozen ... 1. Learn a small set of prefix embeddings. 2. Concatenate them with the input embeddings. 3. Pass through the fixed Transformer." The diagram feeds the learnable prefix in at the embedding layer, alongside the input tokens, into a frozen (lock-icon) Transformer.

Correction: what is described and diagrammed here is prompt tuning, not prefix tuning, despite the slide's title.

- Prefix tuning (Li and Liang, 2021) inserts trainable prefix parameters at **every layer** of the model, not just the input embeddings: "the prefix parameters are inserted in all of the model layers, whereas prompt tuning only adds the prompt parameters to the model input embeddings" (Hugging Face PEFT soft-prompts guide). It also trains the prefixes indirectly through a separate feed-forward network for stability, which is discarded after training.
- Prompt tuning (Lester et al., 2021) is the simpler method the slide actually describes: a small number of learnable "soft token" embeddings prepended to the input only, with everything else frozen. It

"closes the gap" with full fine-tuning once models reach billions of parameters.
- LoRA, mentioned only in this slide's follow-up, is different again: it adds no extra tokens at all, instead injecting trainable low-rank weight-update matrices into existing layers, with no added inference latency (Hu et al., LoRA).
- The slide's stated disadvantage, "slightly lower performance than full fine-tuning", is only partly true: the prefix-tuning paper reports it "obtains comparable performance in the full data setting" and can outperform full fine-tuning in low-data settings, using about 0.1% of the parameters.

Advantages (still valid): far fewer trainable parameters, lower memory/compute, faster training, good for multi-task setups.

Follow-up: how to choose prefix length; can it be combined with other methods (e.g. LoRA)?

Q42. How does the Transformer handle variable-length sequences? What are the common techniques used and their trade-offs?

Third appearance of the variable-length/padding question (see Q27, Q39). Adds a techniques table:

Method	Pros	Cons
Padding + mask	simple, effective	wastes compute on padding
Dynamic batching	more efficient	more complex implementation
Packed sequences	no padding needed	not supported in all frameworks

Correction: repeats the same "Transformers require fixed-length input" framing as Q27; see that correction (padding is a batching convenience, not an architectural requirement). The trade-off table itself (padding+mask vs dynamic batching vs packed sequences) is a reasonable summary and needs no correction.

Q43. What are the main differences between BERT and GPT? When would you use each of them?

Feature	BERT	GPT
Architecture	encoder-only	decoder-only
Attention	bidirectional	causal (unidirectional)
Training objective	masked LM	next-token prediction
Best for	understanding (classification, NER, QA)	generation (chat, summarization, code)
Context usage	both left and right	only left (past)

Interview tip (as printed): "BERT understands context, GPT generates text. The choice depends on the task."

Follow-up: can we use BERT for generation? Can we use GPT for classification? What are the trade-offs?

Correction:

- Missing: BERT was pre-trained with two objectives, not one - masked language modelling plus Next Sentence Prediction (50% actual next sentence, 50% random), per Devlin et al. 2018, section 3.1.
- The framing "BERT understands, GPT generates" is dated: decoder-only models handle classification and QA well through prompting and few-shot learning, which is the central finding of "Language Models are Few-Shot Learners" (GPT-3). The strict either/or framing overstates the practical difference today.

Q44. Positional encoding worked example (exact wording not available - see note)

The slide-by-slide transcription provided for this task was cut off before reaching this question's exact printed wording; the following is reconstructed only from the fact-check task's description of this slide's content: a worked sinusoidal positional-encoding example at d_model = 6, plus a repeat of the "enables better generalization to longer sequences" claim seen on Q3, Q10, Q16, Q36 and Q38.

Correction: the worked numeric table on this slide contains arithmetic errors. Recomputed directly from the deck's own sinusoidal formula (PE(pos,2i) = sin(pos/10000^(2i/d_model)), PE(pos,2i+1) = cos(pos/10000^(2i/d_model))):

Position	Column	Slide's value	Correct value
1	Dim 2	0.010	0.046
1	Dim 4	0.001	0.002
2	Dim 2	0.020	0.093
2	Dim 3	0.999	0.996
2	Dim 4	0.002	0.004

Dims 0 and 1 on this table are correct as printed. See Q3 for the correction to "enables better generalization to longer sequences" - the original paper only hoped for this, and measured extrapolation is weak in practice.

Q45. Beam search (exact wording not available - see note)

As with Q44, the exact printed wording of this slide was not available in the material provided for this task; the following is reconstructed from the fact-check task's description of what the slide claimed: beam search keeps several candidate hypotheses ("keep top-k candidates") at each decoding step, scored by summed log-probabilities to avoid numeric underflow, continuing "until eos", and described as "works well for longer sequences" and "widely used in ... text generation".

Correction:

- Calling the beam width "k" invites confusion with top-k sampling, which this slide's own follow-up also mentions as a separate idea. The standard term is beam width or `num_beams`.
- Length normalization is missing entirely. Summed log-probabilities are negative and grow more negative with every extra token, so unnormalized beam search is biased toward short outputs. The standard fix divides the score by length^alpha - Hugging Face's `length_penalty` is "applied as an exponent to the sequence length ... `length_penalty > 0.0` promotes longer sequences". The original Transformer used beam width 4 with alpha = 0.6 (Vaswani et al. 2017, section 6.1).
- "Works well for longer sequences" and "widely used in text generation" overstate its role in modern generation. Hugging Face's own guidance is that beam search "is best suited for input-grounded tasks, like describing an image or speech recognition" - not open-ended generation. Maximizing likelihood with beam search tends to produce "bland and strangely repetitive" text in open-ended settings (Holtzman et al., "The Curious Case of Neural Text Degeneration"), which is why chat-style LLMs mostly sample instead (temperature, top-p).
- "Repeat until eos" is looser than real implementations: in practice a hypothesis that emits an end-of-sequence token is set aside as finished, and search continues until enough hypotheses are finished or a max length is reached.

What this deck does not cover

The cover slide advertises Vision, Speech, Multimodal, Optimization, Hyperparameters, Training Process and Inference as topics, but the 45 questions barely touch most of them. In a current interview, expect these to come up even though the deck does not prepare you for them:

- **Vision Transformers (ViT).** Only a passing "vision" mention (Q40). Know patch embeddings and the [CLS] token. https://arxiv.org/abs/2010.11929
- **Speech.** Only "CNN for speech" in Q40. Whisper is the reference encoder-decoder speech model. https://arxiv.org/abs/2212.04356
- **Multimodal models.** Only "image captioning" is named, under cross-attention (Q34). CLIP-style contrastive image-text training is the standard example. https://arxiv.org/abs/2103.00020
- **Optimizer, learning-rate warm-up, training schedule.** Not covered at all. Warm-up and why Post-LN needs it (and Pre-LN does not): https://arxiv.org/abs/2002.04745
- **Sampling-based decoding** (temperature, top-k, top-p). Only beam search (Q45, and incompletely) is covered. https://arxiv.org/abs/1904.09751
- **KV caching**, the main memory bottleneck in autoregressive inference at scale. Not mentioned. https://huggingface.co/docs/transformers/kv_cache
- **FlashAttention**, named but mis-described in Q37 as a bandwidth optimization rather than an exact, IO-aware, tiled algorithm. https://arxiv.org/abs/2205.14135
- **RoPE (rotary position embedding).** Only named in a follow-up question (Q37, Q38), never explained. https://arxiv.org/abs/2104.09864
- **Multi-Query and Grouped-Query Attention (MQA/GQA)**, the standard way to shrink the KV cache. Not mentioned. https://arxiv.org/abs/2305.13245
- **RMSNorm.** Only named in a follow-up (Q35), never explained. https://arxiv.org/abs/1910.07467

- **SwiGLU and other gated-linear-unit FFN variants**, now more common than the plain ReLU/GELU FFN the deck describes. https://arxiv.org/abs/2002.05202
- **Mixture of Experts (MoE).** Not mentioned at all. https://arxiv.org/abs/2101.03961
- **LoRA**, only named in a Q41 follow-up. The default parameter-efficient fine-tuning method in current practice. https://arxiv.org/abs/2106.09685
- **Length extrapolation methods (ALiBi and relatives)**, the actual fix for the sinusoidal-encoding extrapolation claim the deck repeats but never substantiates. https://arxiv.org/abs/2108.12409

Is there a free official PDF of these 45 questions?

No. The post's own call to action is "Want the PDF? Comment 'PDF' below!" - that is a comment-gated DM funnel, not a link to a document, and what actually comes back through it (a free file, an email-capture form, or a paid group) cannot be checked without commenting from the owner's own account. The creator's Instagram profile (@techyy_bandaa) returned only Instagram's login wall, so its bio and any link-in-bio could not be read either, and no separate site, Linktree, or repo for this handle turned up. No free official PDF of this deck was found anywhere. This study document is the substitute: it already has everything the deck's slides contain, corrected, in one file.

Sources

Primary source (the deck itself):

- https://www.instagram.com/p/DddmSMCCKbq/ - the Instagram carousel post being fact-checked
- https://www.instagram.com/techyy_bandaa/ - creator's profile (returned only Instagram's login wall when checked; bio and any link-in-bio could not be read)

Papers and documentation used to check and correct the slides:

- https://arxiv.org/abs/1706.03762 and https://arxiv.org/html/1706.03762v7 - Attention Is All You Need (scaling footnote, positional encoding, LayerNorm placement, dropout, Table 1 complexity, masking, multi-head cost, beam width 4 / length penalty 0.6)
- https://arxiv.org/abs/1810.04805 and https://arxiv.org/html/1810.04805 - BERT (MLM 15% / 80-10-10, Next Sentence Prediction, learned position embeddings, bidirectional attention)
- https://arxiv.org/abs/2005.14165 - Language Models are Few-Shot Learners (GPT-3)
- https://arxiv.org/abs/1910.10683 - T5 (Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer)
- https://arxiv.org/abs/2101.00190 - Prefix-Tuning: Optimizing Continuous Prompts for Generation
- https://arxiv.org/abs/2104.08691 - The Power of Scale for Parameter-Efficient Prompt Tuning
- https://huggingface.co/docs/peft/main/en/conceptual_guides/prompting - Hugging Face PEFT, soft prompts (prompt tuning vs prefix tuning vs P-tuning)
- https://arxiv.org/abs/2106.09685 - LoRA: Low-Rank Adaptation of Large Language Models
- https://arxiv.org/abs/2002.04745 - On Layer Normalization in the Transformer Architecture (Pre-LN vs Post-LN)

- https://arxiv.org/abs/2108.12409 and https://arxiv.org/html/2108.12409 - Train Short, Test Long: ALiBi (sinusoidal/rotary/T5-bias extrapolation results)
- https://arxiv.org/abs/2203.16634 - Transformer Language Models without Positional Encodings Still Learn Positional Information
- https://docs.pytorch.org/docs/2.14/generated/torch.nn.functional.scaled_dot_product_attention.html - PyTorch SDPA reference implementation and mask semantics
- https://docs.pytorch.org/docs/2.14/generated/torch.nn.MultiheadAttention.html - PyTorch MultiheadAttention (head-splitting, mask polarity)
- https://docs.pytorch.org/docs/2.14/generated/torch.nn.LayerNorm.html - PyTorch LayerNorm (formula, per-element affine parameters)
- https://docs.pytorch.org/docs/2.14/generated/torch.nn.Dropout.html - PyTorch Dropout (inverted dropout, train vs eval)
- https://huggingface.co/docs/transformers/generation_strategies - Hugging Face generation strategies (beam search use cases)
- https://huggingface.co/docs/transformers/main_classes/text_generation - Hugging Face GenerationConfig (length_penalty)
- https://arxiv.org/abs/1904.09751 - The Curious Case of Neural Text Degeneration
- https://huggingface.co/docs/transformers/kv_cache - Hugging Face Transformers, KV cache strategies
- https://arxiv.org/abs/2205.14135 - FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness
- https://arxiv.org/abs/2010.11929 - An Image is Worth 16x16 Words (ViT)
- https://arxiv.org/abs/2212.04356 - Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)
- https://arxiv.org/abs/2103.00020 - Learning Transferable Visual Models From Natural Language Supervision (CLIP)
- https://arxiv.org/abs/2104.09864 - RoFormer: Enhanced Transformer with Rotary Position Embedding (RoPE)
- https://arxiv.org/abs/2305.13245 - GQA: Training Generalized Multi-Query Transformer Models
- https://arxiv.org/abs/1910.07467 - Root Mean Square Layer Normalization (RMSNorm)
- https://arxiv.org/abs/2002.05202 - GLU Variants Improve Transformer (SwiGLU)
- https://arxiv.org/abs/2101.03961 - Switch Transformers (Mixture of Experts)